**Universidad de Caldas**

**Facultad de Inteligencia Artificial e Ingenierías**

# Búsqueda de la configuración de prompt bajo presupuesto de consultas: formalización, NP-dureza y solución base del Proyecto Oráculo

Jackeline Rivera Aguilera

Facultad de Inteligencia Artificial e Ingenierías, Universidad de Caldas

Análisis y diseño de algoritmos – Grupo 1

Docente: Luz Enith Guerrero Mendieta

02 de Noviembre de 2026

**Resumen**

El Proyecto Oráculo plantea elegir, para un modelo de lenguaje fijo (Qwen3-1.7B), una configuración de prompt, un índice entre ocho opciones para cada una de cinco ranuras de texto instruccional, más la temperatura de generación que maximice la fracción de peticiones cuyas restricciones de formato verificables se cumplen todas.

Este informe formaliza el problema en dos partes: la configuración óptima con un oráculo exacto y la búsqueda con consultas limitadas y ruidosas. Para la primera se demuestra la NP-completitud de su versión de decisión mediante una reducción polinomial desde 3-SAT que conserva el número de aciertos, de modo que la versión de optimización hereda además el límite de aproximación de MAX-3SAT. Para la segunda se analizan el costo de evaluación y la complejidad de muestreo. La solución base combina un cribado por ranura guiado por las trazas de fallos, una poda, un ascenso por coordenadas, una verificación del vecindario completo y una prueba de temperatura. Con 103 evaluaciones (10 300 consultas, el 0,31 % de las que exige la fuerza bruta) encontró la configuración rol 6, estrategia 6, formato 0, estilo 7, cierre 3 y temperatura 0.0, con 21 % de precisión en búsqueda frente al 16 % sin instrucciones y 18,4 % en validación. La configuración es un óptimo local verificado, no un óptimo global demostrado; la interacción entre ranuras observada explica por qué.

*Palabras clave:* optimización de prompts, NP-completitud, reducción desde 3-SAT, ascenso por coordenadas, complejidad de muestreo

**Tabla de contenido**

Introducción ..... 6
Planteamiento del problema ..... 7
Contexto y decisiones del sistema ..... 7
Espacio de alternativas, lote de búsqueda y métrica ..... 7
Diferencias con el enunciado general ..... 8
Formalización ..... 8
Notación ..... 8
Evaluación de una configuración ..... 9
Tamaño de la entrada ..... 10
Parte 1: configuración óptima con oráculo exacto ..... 10
Parte 2: búsqueda con consultas limitadas y ruidosas ..... 11
Supuestos y alcance ..... 11
Dificultad computacional ..... 12
Problemas de partida ..... 12
Construcción de la reducción ..... 13
La transformación es polinomial ..... 14
Lema de conteo ..... 14
Equivalencia de respuestas ..... 15
Pertenencia a NP y conclusión ..... 15
Parte 2: costo de evaluación y complejidad de muestreo ..... 15
Método de optimización ..... 16
Justificación del método ..... 16
Fases, criterios y condiciones de parada ..... 17
Parámetros, presupuesto y costo ..... 18
Relación con el cuaderno de búsqueda ..... 19
Resultados ..... 19
Línea base y cribado ..... 19
Uso de las trazas de fallos ..... 20
Ascenso y verificación ..... 22
Óptimo local verificado ..... 23
Temperatura ..... 24
Configuración final y costo ..... 25
Validación ..... 25
Discusión y limitaciones ..... 26
Reproducibilidad ..... 27
Conclusiones ..... 28

Referencias.....30

## Lista de tablas

Tabla 1. Diferencias entre el enunciado general y la versión del problema usada

Tabla 2. Notación y valores en el proyecto

Tabla 3. Correspondencia entre 3-SAT y PROMPT-DEC

Tabla 4. Instancias generadas para la cláusula $C_1 = (x_1 \vee \neg x_2 \vee x_3)$

Tabla 5. Fases de la búsqueda

Tabla 6. Resultado de la poda guiada por trazas

Tabla 7. Trayectoria de las fases 2 y 3 (movimientos aceptados)

Tabla 8. Precisión de la configuración elegida según la temperatura

Tabla 9. Resumen de resultados

Tabla 10. Artefactos del repositorio

## Lista de figuras

Figura 1. Flujo de la solución base

Figura 2. Precisión de cada opción evaluada sola en el cribado

Figura 3. Precisión de cada evaluación y mejor precisión hasta el momento

Figura 4. Vecindario de la configuración elegida

## Introducción

Un modelo de lenguaje no responde igual a una misma petición si se le antepone o pospone texto instruccional distinto. El Proyecto Oráculo del curso Análisis y Diseño de Algoritmos convierte esa observación en un problema de optimización discreta: elegir, de un catálogo de cinco ranuras con ocho opciones cada una, la combinación de instrucciones y la temperatura de generación que maximiza la fracción de peticiones cuyas restricciones verificables se cumplen todas (DanielMelo404, 2026; Universidad de Caldas, 2026a). La dificultad es doble: el número de combinaciones crece exponencialmente con el número de ranuras y cada evaluación cuesta tiempo de GPU, cerca de tres minutos por cada 100 peticiones en una tarjeta NVIDIA T4.

Este informe corresponde a la primera entrega del Grupo G01 (Universidad de Caldas, 2026b) y cubre tres productos: la formalización de las dos partes del problema; la prueba de NP-dureza de la primera parte por reducción desde 3-SAT, junto con el análisis de costo y de muestreo de la segunda; y la solución base, con el proceso de optimización que efectivamente se ejecutó y que produjo el archivo `entrega.json`. El código, los cuadernos de búsqueda y de evaluación y el manual técnico están en el repositorio del grupo (ver la sección Reproducibilidad).

El resultado principal es la configuración rol 6, estrategia 6, formato 0, estilo 7, cierre 3 y temperatura 0.0, que alcanzó 21 % de precisión en las 100 instancias de búsqueda (16 % sin instrucciones) y 18,4 % en las 87 de validación. Se obtuvo con 103 evaluaciones, el 0,31 % de las consultas que exigiría probar todas las configuraciones, y es un óptimo local verificado: ninguna configuración que cambie una sola ranura la supera. No es, ni se presenta como, un óptimo global demostrado.

## Planteamiento del problema

### Contexto y decisiones del sistema

El modelo es Qwen3-1.7B (Qwen Team, 2025) con el modo de razonamiento desactivado y no se reentrena. Cada petición proviene de un conjunto de instrucciones con restricciones verificables del tipo de IFEval e IFBench (Pyatkin et al., 2025; Zhou et al., 2023): por ejemplo, «la última palabra de cada oración debe ser *officer*», «no use comas» o «incluya un título entre dobles ángulos». Un verificador automático decide si la respuesta cumple cada restricción.

Lo que el sistema decide es qué texto del catálogo ocupa cada ranura y con qué temperatura se genera. El prompt se arma en un orden fijo: rol, estrategia, formato, estilo, la petición y, al final, el cierre. Según el diseño del catálogo, cada ranura actúa sobre una parte distinta de la respuesta: el rol sobre el prefijo, la estrategia sobre el largo, el formato sobre la estructura, el estilo sobre los caracteres y el cierre sobre el sufijo. La opción 3 del cierre es la única que usa información de la instancia: repite al final el texto de sus restricciones. El recurso que se quiere cuidar es el tiempo de GPU, medido en consultas: una consulta es una petición generada y verificada con una configuración.

### Espacio de alternativas, lote de búsqueda y métrica

Con temperatura fija en 0.0 hay $8^5 = 32\,768$ configuraciones; con las temperaturas 0.0, 0.3 y 0.7 hay 98 304. El archivo de datos visibles trae 450 instancias: 100 de búsqueda, 87 de validación y 263 descartadas por el curso porque no informaban (imposibles de acertar, imposibles de fallar, con una sola restricción o con cuatro o cinco). Las instancias de búsqueda y de validación tienen en promedio 2,5 y 2,4 restricciones. La métrica es la precisión todo-o-nada: la fracción de instancias en que la respuesta cumple todas sus restricciones.

### Diferencias con el enunciado general

El enunciado general del proyecto (Universidad de Caldas, 2026a) describe una versión anterior del problema. La guía de la primera entrega del Grupo 1 y el repositorio oficial fijan los valores que se usan en este informe. La Tabla 1 resume las diferencias.

**Tabla 1**

*Diferencias entre el enunciado general y la versión del problema usada*

| Aspecto | Enunciado general | Repositorio y guía del G1 (usado aquí) |
|---|---|---|
| Ranuras | Rol, Estrategia, Formato, Verificación, Recordatorio | rol, estrategia, formato, estilo, cierre (verificación y recordatorio se fundieron en el cierre) |
| Opciones por ranura | k_max = 4 | 8 |
| Tamaño del espacio | \|C\| = 3840 | $8^5$ = 32 768 (98 304 con tres temperaturas) |
| Instancias de búsqueda | n = 150 | 100 y 87 de validación |
| Reducción pedida | Desde Cobertura Máxima | Desde 3-SAT configuración óptima con oráculo exacto |

*Nota.* Además, la cuenta del enunciado para Successive Halving, $3840 \cdot 2 \cdot \lceil \log_2 3840 \rceil = 91\,444$, no es exacta: el producto es 92 160.

## Formalización

### Notación

Para que el tamaño del problema pueda crecer, se formaliza con $m$ ranuras y $k_s$ opciones por ranura; el caso del curso es $m = 5$ y $k_s = 8$. La Tabla 2 reúne la notación.

**Tabla 2**

*Notación y valores en el proyecto*

| Símbolo | Significado | Valor en el proyecto |
|---|---|---|
| $S = \{s_1, \ldots, s_m\}$ | Ranuras del prompt | m = 5 |
| $O_s$, $k_s = \|O_s\|$ | Opciones de la ranura s | $k_s = 8$ |

| Símbolo | Significado | Valor en el proyecto |
|---|---|---|
| T | Temperaturas permitidas | {0.0} o {0.0, 0.3, 0.7} |
| $C = O_1 \times \ldots \times O_{\square} \times T$ | Espacio de configuraciones | $\|C\| = 32\,768$ (o 98 304) |
| $c = (o_1, \ldots, o_{\square}, \tau)$ | Una configuración | p. ej. (6, 6, 0, 7, 3, 0.0) |
| $I = \{i_1, \ldots, i_{\square}\}$ | Instancias de búsqueda | n = 100 |
| V | Instancias de validación | \|V\| = 87 |
| $p_i$, $R_i$ | Petición y restricciones de la instancia i | $\ell_i = \|R_i\| \in \{2, 3\}$ |
| $v_\rho : \Sigma^* \to \{0, 1\}$ | Verificador de la restricción ρ | IFEvalG / IFBench |
| $M_\tau$ | Modelo que genera con temperatura τ | Qwen3-1.7B, hasta L = 384 tokens |
| B | Presupuesto de consultas | Parte 2 |

**Evaluación de una configuración**

El oráculo arma el prompt concatenando el texto de las ranuras y la petición. Las ranuras ausentes no aportan texto:

$$\pi(c, i) = o_{\text{rol}} \,\|\, o_{\text{estrategia}} \,\|\, o_{\text{formato}} \,\|\, o_{\text{estilo}} \,\|\, p_i \,\|\, o_{\text{cierre}}(i) \quad (1)$$

La instancia se acierta solo si la respuesta cumple todas sus restricciones; el producto expresa el todo-o-nada:

$$\text{éxito}(c, i) = \prod_{\rho \in R_i} v_\rho(\, M_\tau(\, \pi(c, i)\,)\,) \in \{0, 1\} \quad (2)$$

y la precisión de *c* sobre un conjunto de instancias *I* es

$$\text{Acc}_I(c) = (1/n) \cdot \Sigma_{i \in I}\ \text{éxito}(c, i) \quad (3)$$

**Tamaño de la entrada**

El tamaño no se reduce a una sola variable. Crecen el número de ranuras *m* y de opciones *k*, que determinan $|C| = k^m \cdot |T|$; el número de instancias *n*, que multiplica el costo de evaluar cada configuración; y el número de restricciones por instancia ℓ, que hace más difícil acertar. Una descripción explícita del problema ocupa del orden de $m \cdot k + n \cdot \ell$ símbolos (más la

descripción del oráculo), mientras que el espacio de búsqueda tiene $k^m$ elementos: es **exponencial en el tamaño de la entrada**.

Por eso es necesario generalizar a $m$ ranuras. Con $m = 5$ y $k = 8$ fijos, el problema tiene tamaño constante y se resuelve en tiempo constante probando las 32 768 configuraciones; hablar de NP-dureza solo tiene sentido para la familia de problemas en la que $m$ crece.

**Parte 1: configuración óptima con oráculo exacto**

En esta parte se supone que el oráculo es exacto y gratuito. Con $\tau = 0$ la generación es determinista o decodificación voraz, de modo que éxito($c$, $i$) es una función fija. Para la prueba de dificultad se usa un modelo de compatibilidad: para cada instancia $i$ y ranura $s$ hay un conjunto $A_{i,\square} \subseteq O_\square$ de opciones admisibles, y la instancia se acierta si y solo si $o_\square \in A_{i,\square}$ para todas las ranuras. El modelo captura el comportamiento observado, una opción como «empiece cada respuesta con *Happy to help!*» es incompatible con una restricción sobre la primera palabra y es un caso particular del problema general: si el caso particular es NP-difícil, el general también lo es.

- **Entrada:** las ranuras S, los catálogos $O_\square$, las temperaturas T, las instancias I y los conjuntos admisibles $A_{i,\square}$.
- **Salida:** una configuración $c^* \in C$.
- **Restricciones:** exactamente una opción por ranura y $\tau \in T$.
- **Objetivo:** maximizar la precisión,

$$c^* = \operatorname{argmax}_{c \in C} \mathrm{Acc}_I(c) \quad (4)$$

**Versión de decisión (PROMPT-DEC).** Dado además un entero $\theta$, ¿existe $c \in C$ tal que $\Sigma_{i \in I}$ éxito($c$, $i$) $\geq \theta$? Se usa el número de aciertos en lugar de la fracción porque es equivalente (precisión $\geq \theta/n$) y simplifica las cuentas.

**Parte 2: búsqueda con consultas limitadas y ruidosas**

Este es el escenario que se ejecutó: el oráculo es una caja negra costosa.

- **Costo:** cada consulta ($c$, $i$) genera hasta 384 tokens en una GPU T4. El caché del oráculo hace que repetir la misma combinación de modelo, configuración, instancia y semilla no cueste nada.
- **Ruido:** $\mathrm{Acc}_I(c)$ es un estimador de la precisión sobre la distribución de la que salen las instancias del conjunto privado, $\mathrm{Acc}_D(c)$. Con $\tau > 0$ se suma la aleatoriedad del muestreo de tokens.
- **Entrada:** C, I, V y un presupuesto B de consultas.
- **Salida:** una estrategia de consultas cuántas consultas $m$□ recibe cada configuración $c$□ y una configuración elegida $\hat{c}$.
- **Objetivo y presupuesto:**

maximizar $\mathrm{E}[\ \mathrm{Acc}_D(\hat{c})\ ]$ sujeto a $\Sigma_{j=1}^{|C|}\, m\square \leq B(5)$

- **Restricción metodológica:** se busca solo sobre I; V se usa una sola vez para validar la configuración ya elegida.

**Supuestos y alcance**

- El modelo, el catálogo y los verificadores son fijos; con $\tau = 0$ el oráculo es determinista.
- Las instancias de búsqueda y de validación provienen de la misma distribución; el conjunto privado puede incluir familias de restricciones nuevas.
- El modelo de compatibilidad simplifica las interacciones reales entre ranuras; se usa solo para la prueba de dificultad.

- Esta entrega desarrolla la solución base. El algoritmo exacto, Successive Halving, la comparación de repartos de presupuesto y la validación experimental con instancias sintéticas corresponden a entregas posteriores.

## Dificultad computacional

Esta sección demuestra que la Parte 1 es NP-completa en su versión de decisión, mediante una reducción polinomial desde 3-SAT (Cook, 1971; Garey & Johnson, 1979; Karp, 1972). La demostración supone un oráculo exacto y no se traslada sin más a la Parte 2, que se analiza al final por costo de evaluación y complejidad de muestreo.

### Problemas de partida

**3-SAT.** Entrada: una fórmula φ en forma normal conjuntiva sobre las variables $x_1, \ldots, x_N$ con cláusulas $C_1, \ldots, C_K$, cada una con exactamente tres literales sobre tres variables distintas. Pregunta: ¿existe una asignación de verdad que satisfaga todas las cláusulas? Es NP-completo (Cook, 1971; Karp, 1972). La condición de variables distintas no resta generalidad: una cláusula con una variable repetida se simplifica o se rellena con variables nuevas en tiempo polinomial.

**PROMPT-DEC.** Entrada: $m$ ranuras con catálogos $O_1, \ldots, O_\square$; $n$ instancias; para cada instancia $i$ y ranura $s$, un conjunto $A_{i,\square} \subseteq O_\square$; y un entero θ. La instancia $i$ se acierta con $c$ si y solo si $o_\square \in A_{i,\square}$ para toda $s$. Pregunta: ¿existe $c$ con al menos θ aciertos?

### Construcción de la reducción

Para demostrar que PROMPT-DEC es difícil, se transforma cualquier instancia de 3-SAT en una instancia de PROMPT-DEC (3-SAT $\leq_\square$ PROMPT-DEC); la dirección importa. La correspondencia que sostiene la prueba está en la Tabla 3.

**Tabla 3**

*Correspondencia entre 3-SAT y PROMPT-DEC*

| 3-SAT | PROMPT-DEC |
|---|---|
| Variable booleana x□ | Ranura j con opciones {V, F} |
| Asignación de verdad | Configuración |
| Cláusula satisfecha | Exactamente una de sus siete instancias acertada |
| ¿Se satisfacen las K cláusulas? | ¿Hay una configuración con al menos θ = K aciertos? |

Dada φ, se construye:

1. **Ranuras.** Una por variable: $m$ = N y $O$□ = {V, F}.
2. **Instancias.** Sea $C_q$ una cláusula sobre las variables $x_a$, $x_b$, $x_c$. De sus $2^3 = 8$ asignaciones parciales, exactamente siete la satisfacen (solo falla la que hace falsos los tres literales). Por cada asignación α que la satisface se crea la instancia $i_{q,\alpha}$ con $A_{i,a} = \{\alpha(a)\}$, $A_{i,b} = \{\alpha(b)\}$, $A_{i,c} = \{\alpha(c)\}$ y $A_{i,s} = \{V, F\}$ en las demás ranuras.
3. **Umbral.** θ = K.

Por ejemplo, para $C_1 = (x_1 \vee \neg x_2 \vee x_3)$ la única asignación que falla es $x_1$ = F, $x_2$ = V, $x_3$ = F, y se generan las siete instancias de la Tabla 4.

**Tabla 4**

*Instancias generadas para la cláusula $C_1 = (x_1 \vee \neg x_2 \vee x_3)$*

| Instancia | A($x_1$) | A($x_2$) | A($x_3$) | Demás ranuras |
|---|---|---|---|---|
| $i_{1,1}$ | {V} | {V} | {V} | {V, F} |
| $i_{1,2}$ | {V} | {V} | {F} | {V, F} |
| $i_{1,3}$ | {V} | {F} | {V} | {V, F} |
| $i_{1,4}$ | {V} | {F} | {F} | {V, F} |
| $i_{1,5}$ | {F} | {V} | {V} | {V, F} |

| Instancia | $A(x_1)$ | $A(x_2)$ | $A(x_3)$ | Demás ranuras |
|---|---|---|---|---|
| $i_{1,6}$ | {F} | {F} | {V} | {V, F} |
| $i_{1,7}$ | {F} | {F} | {F} | {V, F} |

*Nota.* La asignación (F, V, F) no genera instancia: es la única que no satisface $C_1$.

**La transformación es polinomial**

Se crean 7K instancias, cada una con N conjuntos admisibles de tamaño a lo sumo dos. La construcción toma tiempo $O(N \cdot K)$, polinomial en el tamaño de φ.

**Lema de conteo**

**Lema.** Para toda configuración *c*, las siete instancias de la cláusula $C_q$ aportan exactamente un acierto si *c*, leída como asignación de verdad, satisface $C_q$, y ningún acierto en caso contrario.

**Demostración.** La configuración *c* fija un valor para $x_a$, $x_b$ y $x_c$, es decir, coincide con exactamente una de las ocho asignaciones parciales. La instancia $i_{q,\alpha}$ se acierta si y solo si *c* coincide con α en esas tres ranuras, porque en las demás todo es admisible. Si la asignación de *c* satisface $C_q$, es una de las siete y se acierta exactamente una instancia; si no, es la asignación excluida y no se acierta ninguna.

**Consecuencia.** Para toda configuración, el número total de aciertos es igual al número de cláusulas que satisface.

**Equivalencia de respuestas**

**(⇒)** Si φ es satisfacible con la asignación σ, la configuración $c = \sigma$ satisface las K cláusulas y, por el lema, logra $K \geq \theta$ aciertos: la respuesta de PROMPT-DEC es sí.

**(⇐)** Si existe *c* con al menos K aciertos, como cada cláusula aporta a lo sumo uno, *c* satisface las K cláusulas, y la asignación $\sigma = c$ satisface φ: la respuesta de 3-SAT es sí.

**Pertenencia a NP y conclusión**

PROMPT-DEC está en NP: dada una configuración, contar sus aciertos toma O($n \cdot m$) operaciones. Con la reducción anterior, PROMPT-DEC es NP-completo y la versión de optimización (encontrar $c^*$) es NP-difícil, porque resolverla en tiempo polinomial permitiría responder la versión de decisión comparando su valor con θ.

La prueba se mantiene con el catálogo real de ocho opciones: basta agregar a cada ranura seis opciones de relleno que ninguna instancia admite, que nunca aportan aciertos. Con la temperatura fija en 0.0, la dimensión T no cambia nada.

Hay una consecuencia adicional. La reducción conserva exactamente el valor objetivo aciertos = cláusulas satisfechas y la correspondencia entre soluciones es la identidad, de modo que también reduce MAX-3SAT a la versión de optimización. Como MAX-3SAT no admite un algoritmo polinomial con razón de aproximación mejor que 7/8 salvo que P = NP (Håstad, 2001), lo mismo vale para la configuración óptima con oráculo exacto. Esto justifica buscar heurísticas con buen desempeño empírico en lugar de garantías de optimalidad.

**Parte 2: costo de evaluación y complejidad de muestreo**

**Costo de la fuerza bruta.** Probar todas las configuraciones sobre las 100 instancias exige $|C| \cdot n = 32\,768 \times 100 = 3\,276\,800$ consultas. Con el tiempo medido en este proyecto, unos 180 segundos por evaluación de 100 instancias, serían 98 304 minutos: cerca de 68 días de GPU continua.

**Complejidad de muestreo.** Aun con un oráculo determinista, la precisión observada en $n$ instancias estima la precisión sobre la distribución. Con precisiones cercanas al 20 %, el error estándar con $n = 100$ es $\sqrt{(0{,}2 \cdot 0{,}8 / 100)} \approx 4$ puntos porcentuales. Sin supuestos de distribución,

la desigualdad de Hoeffding (1963) da el número de instancias necesario para que la precisión estimada esté a menos de ε de la real con probabilidad al menos $1 - \delta$:

$$n \geq \ln(2/\delta) / (2\varepsilon^2) \quad (6)$$

Con $\varepsilon = \delta = 0{,}05$ se necesitan 738 instancias, más de las 100 disponibles. Si además se quiere que la garantía valga a la vez para las $M$ configuraciones comparadas, la cota de la unión lleva a

$$n \geq \ln(2M/\delta) / (2\varepsilon^2) \quad (7)$$

que con las $M = 103$ evaluaciones de este proyecto exige 1664 instancias. Leído al revés, con $n = 100$ el margen garantizado para una sola configuración es $\varepsilon \approx 0{,}136$. Por eso las diferencias de uno o dos puntos entre configuraciones no son concluyentes, y por eso este informe habla de la mejor configuración **observada**. Comparar todas las configuraciones sobre las mismas instancias (comparación pareada) reduce la varianza de las diferencias, y la validación sobre instancias no usadas en la búsqueda es la comprobación externa.

## Método de optimización

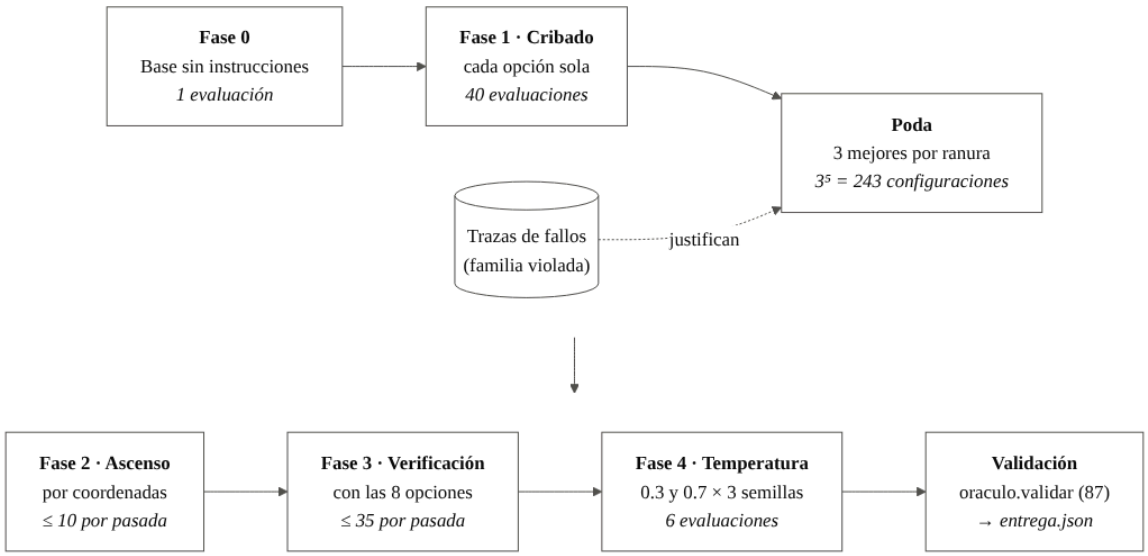
### Justificación del método

La fuerza bruta es inviable y una búsqueda aleatoria rinde poco: según el repositorio del curso, una configuración al azar queda por debajo del 5 % porque en cada ranura la mayoría de opciones daña la respuesta (DanielMelo404, 2026). El método aprovecha la estructura del problema. Como cada ranura actúa sobre una parte distinta de la respuesta, el efecto de cada opción se puede medir por separado y las trazas de fallos que indican qué familia de restricción falló primero  permiten explicar y podar las opciones dañinas. Sobre el espacio reducido se aplica un ascenso por coordenadas, natural en un espacio que es un producto cartesiano (Wright,

2015). Como las ranuras interactúan, el resultado se verifica después con las ocho opciones de cada ranura. La Figura 1 resume el flujo.

**Figura 1**

*Flujo de la solución base*




*Nota.* Cada evaluación usa las mismas 100 instancias de búsqueda con semilla 1. Las trazas de fallos (primera familia de restricción violada) justifican la poda.

## Fases, criterios y condiciones de parada

La Tabla 5 resume las fases. La notación es $m = 5$ ranuras, $k = 8$ opciones, $c = 3$ candidatos que sobreviven por ranura y $P = 3$ pasadas como máximo

**Tabla 5**

*Fases de la búsqueda*

| Fase | Qué hace | Criterio de decisión | Parada | Evaluaciones (máx.) |
|---|---|---|---|---|
| 0 · Base | Prompt sin instrucciones | — | — | 1 |

| Fase | Qué hace | Criterio de decisión | Parada | Evaluaciones (máx.) |
|---|---|---|---|---|
| 1 · Cribado | Cada opción sola (las demás ranuras vacías); mide roturas y arreglos frente a la base, con la familia violada | — | Al recorrer las m·k opciones | 40 |
| Poda | Ordena cada ranura por precisión (desempate: menos roturas) | Conserva las c = 3 mejores; registra las familias rotas de las descartadas | — | 0 |
| 2 · Ascenso | Parte de la mejor opción de cada ranura; en cada ranura prueba sus candidatos con las otras fijas | Cambia solo si la precisión mejora estrictamente | Pasada sin mejoras o P = 3 | $1 + P \cdot m \cdot (c - 1) = 31$ |
| 3 · Verificación | El mismo ascenso con las 8 opciones por ranura | Ídem | Ídem; si converge, óptimo local 1-opt | $P \cdot m \cdot (k - 1) = 105$ |
| 4 · Temperatura | $\tau \in \{0.3, 0.7\}$ con semillas 1, 2 y 3 | Cambia a $\tau > 0$ solo si su promedio supera a $\tau = 0$ | Al terminar | 6 |

*Nota.* Una evaluación son 100 consultas. Las cifras son cotas superiores; el caché evita repetir configuraciones ya evaluadas.

**Terminación.** En las fases 2 y 3 cada cambio aumenta estrictamente la precisión, que solo puede tomar los $n + 1$ valores $0/n$, $1/n$, …, $n/n$; por lo tanto el ascenso no puede ciclar y termina siempre, además del tope de $P$ pasadas. Cuando una pasada completa de la Fase 3 no encuentra mejoras, la configuración es un óptimo local 1-opt: ninguna configuración que difiera en una sola ranura la supera.

**Parámetros, presupuesto y costo**

Todas las configuraciones se miden sobre las mismas 100 instancias de búsqueda con semilla 1, lo que hace pareadas las comparaciones. El oráculo se usó con lotes de hasta 16 respuestas y 384 tokens como máximo por respuesta, y su caché se guardó en Google Drive para que una desconexión de Colab no obligara a repetir lo generado. En el peor caso, con estos parámetros, el método consume

$n \cdot [1 + m \cdot k + P \cdot m \cdot (c - 1) + P \cdot m \cdot (k - 1) + 6] = 100 \cdot 182 = 18\,200$ consultas(8)

es decir, O($n \cdot m \cdot k \cdot P$), polinomial en $m$ y $k$, frente a $n \cdot k^m$ de la fuerza bruta, exponencial en $m$. El trabajo propio del algoritmo (comparar conjuntos, contar familias) es O($n$) por evaluación y despreciable frente a la generación del modelo.

**Relación con el cuaderno de búsqueda**

El procedimiento está implementado en los módulos `busqueda.py` (las fases) y `registro.py` (la bitácora de evaluaciones), y se ejecuta en el cuaderno `busqueda_G01.ipynb`. La ejecución completa, con todas sus salidas e interpretaciones, está en `busqueda_G01_resultado.ipynb`: la sección 4 contiene las fases 0 a 4, la tabla de poda y la lectura de trazas; la sección de registro, la tabla de las 103 evaluaciones y las curvas; y la sección final, la validación y la generación de `entrega.json`.

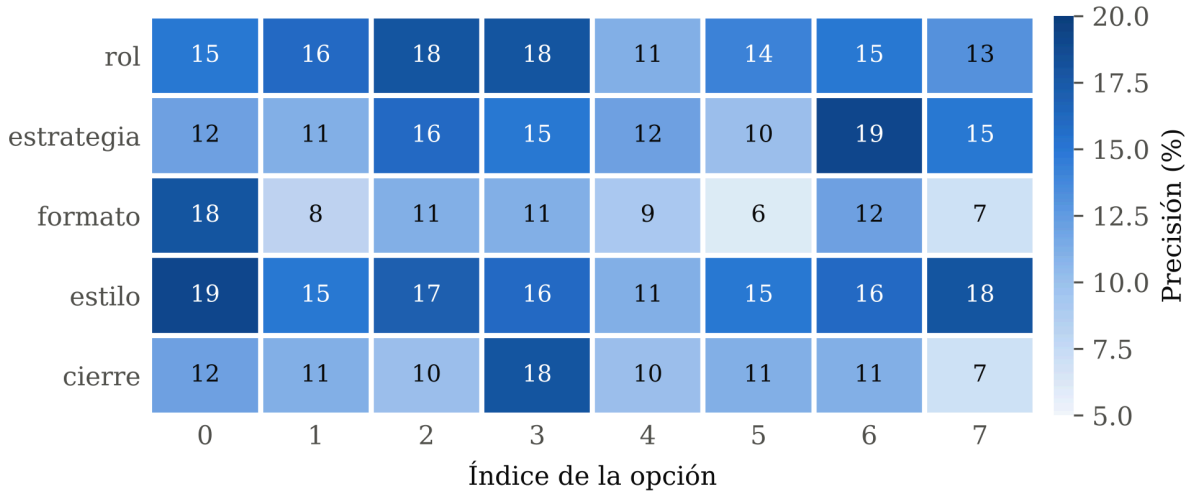
## Resultados

**Línea base y cribado**

Sin ninguna instrucción, el modelo acierta 16 de las 100 instancias de búsqueda. La Figura 2 muestra la precisión de cada una de las 40 opciones evaluada sola. Ninguna supera el 19 % y muchas quedan por debajo de la base: la ranura de formato es la más dañina (de 6 % a 12 % salvo la opción 0) y en el cierre solo la opción 3, el recordatorio de restricciones, mejora la base (18 % y ocho instancias arregladas).

**Figura 2**

*Precisión de cada opción evaluada sola en el cribado*




*Nota.* Precisión en las 100 instancias de búsqueda (%), con las otras cuatro ranuras vacías. Base sin instrucciones: 16 %.

**Uso de las trazas de fallos**

Las trazas registran, para cada instancia fallida, la primera familia de restricción violada y el texto generado. Su lectura llevó a cinco conclusiones:

1. **Las opciones que imponen una forma literal rompen la familia de la parte de la respuesta que tocan.** Los roles 0, 7, 4 y 3 pegan un prefijo fijo (*Happy to help!*, *Sure thing!*, *Topic:*, *Answer:*); el formato 1 (documento en markdown) rompe `number_bullet_lists`; el estilo 4 (todo en minúsculas) rompe `english_capital`; y los cierres que agregan una línea final (0, 4, 5 y 7) rompen sobre todo `last_word_answer`.
2. **Varias opciones hacen que el modelo copie la instrucción del catálogo en su respuesta.** Con el formato 5 y 7, el estilo 1 y 5 y el cierre 2, la salida empieza repitiendo

el texto de la instrucción (por ejemplo, *Number every paragraph of your reply…*). Esto explica que el formato 5 y el 7 sean las peores opciones de todo el cribado (6 % y 7 %).

3. **El recordatorio de restricciones es el único cierre que ayuda**, con 18 % frente a 7 %–12 % de los demás.
4. **Hay instancias frágiles** que casi cualquier instrucción tumba: la 386 (copiar un texto literal) y la 483 (última palabra). Aparecen en las roturas de casi todas las opciones, por lo que el conteo de roturas discrimina menos de lo esperado.
5. **Las diferencias pequeñas no son concluyentes.** Con un error estándar cercano a ±3,7 puntos, muchas opciones difieren de la base dentro del ruido. Por eso la poda conserva tres opciones por ranura en lugar de una, y la Fase 3 vuelve a probar las ocho.

La Tabla 6 resume la poda: de 40 opciones se descartaron 25, y el espacio pasó de 32 768 a $3^5 = 243$ configuraciones para la Fase 2.

**Tabla 6**

*Resultado de la poda guiada por trazas*

| Ranura | Conservadas (precisión) | Descartadas (precisión) | Evidencia principal en las trazas de las descartadas |
|---|---|---|---|
| rol | 3 (18 %), 2 (18 %), 1 (16 %) | 0 (15), 6 (15), 5 (14), 7 (13), 4 (11) | Prefijos fijos; 7: english_lowercase ×2; 4: number_words ×2 |
| estrategia | 6 (19 %), 2 (16 %), 3 (15 %) | 7 (15), 4 (12), 0 (12), 1 (11), 5 (10) | Respuestas infladas: last_word_answer ×2 en 7, 4 y 1 |
| formato | 0 (18 %), 6 (12 %), 3 (11 %) | 2 (11), 4 (9), 1 (8), 7 (7), 5 (6) | 1: number_bullet_lists ×3; 7 y 5: eco de la instrucción (13 y 11 roturas) |
| estilo | 0 (19 %), 7 (18 %), 2 (17 %) | 6 (16), 3 (16), 1 (15), 5 (15), 4 (11) | 4: english_capital ×2; 1 y 5: eco de la instrucción |
| cierre | 3 (18 %), 0 (12 %), 5 (11 %) | 6 (11), 1 (11), 4 (10), 2 (10), 7 (7) | Línea final añadida: last_word_answer ×2–3; 2: eco y number_bullet_lists ×2 |

*Nota.* Precisiones en % sobre las 100 instancias de búsqueda, con la opción sola. Se conservan las tres mejores por ranura; en empate, la de menos roturas.

**Ascenso y verificación**

La Tabla 7 muestra los movimientos aceptados. El punto de partida la mejor opción de cada ranura evaluada sola dio solo 15 %, menos que varias opciones aisladas: combinar lo mejor de cada ranura no produce lo mejor del conjunto. El ascenso de la Fase 2 subió a 19 % y convergió en la segunda pasada.

**Tabla 7**

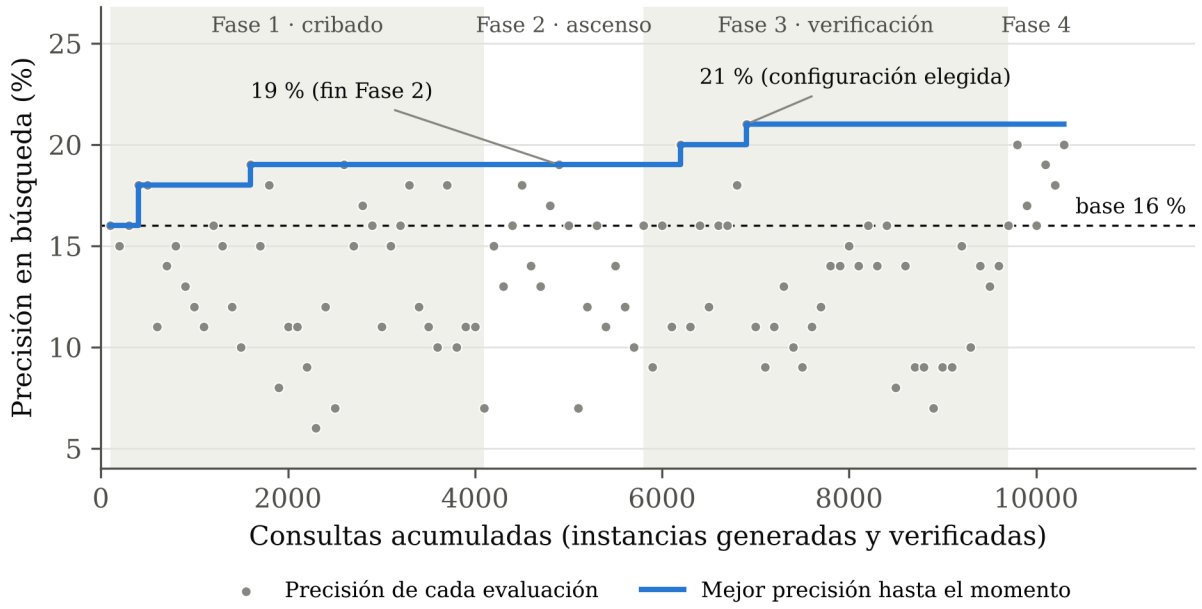
*Trayectoria de las fases 2 y 3, movimientos aceptados*

| Fase y pasada | Cambio | Configuración resultante (rol, estr., form., est., cier.) | Precisión |
|---|---|---|---|
| Inicio | Mejor opción de cada ranura sola | (3, 6, 0, 0, 3) | 15 % |
| Fase 2 · pasada 1 | rol 3 → 1 | (1, 6, 0, 0, 3) | 16 % |
| Fase 2 · pasada 1 | estrategia 6 → 2 | (1, 2, 0, 0, 3) | 18 % |
| Fase 2 · pasada 1 | estilo 0 → 7 | (1, 2, 0, 7, 3) | 19 % |
| Fase 2 · pasada 2 | Sin mejoras: convergió | (1, 2, 0, 7, 3) | 19 % |
| Fase 3 · pasada 1 | rol 1 → 6 (descartado en la poda) | (6, 2, 0, 7, 3) | 20 % |
| Fase 3 · pasada 1 | estrategia 2 → 6 | (6, 6, 0, 7, 3) | 21 % |
| Fase 3 · pasada 2 | Sin mejoras: convergió | (6, 6, 0, 7, 3) | 21 % |

La Fase 3 cambió el resultado. El rol 6 había sido descartado en el cribado, porque solo daba 15 %, es decir, por debajo de la base, y sin embargo es la mejor opción de su ranura cuando se combina con las demás. Algo parecido ocurre con la estrategia 6: con el rol 1 daba 14 %, con el rol 6 da 21 %. Es la interacción entre ranuras el efecto de una opción depende de las otras que hace difícil el problema y que justifica no confiar solo en la poda. La Figura 3 muestra la evolución de la mejor precisión frente a las consultas acumuladas.

**Figura 3**

*Precisión de cada evaluación y mejor precisión hasta el momento*



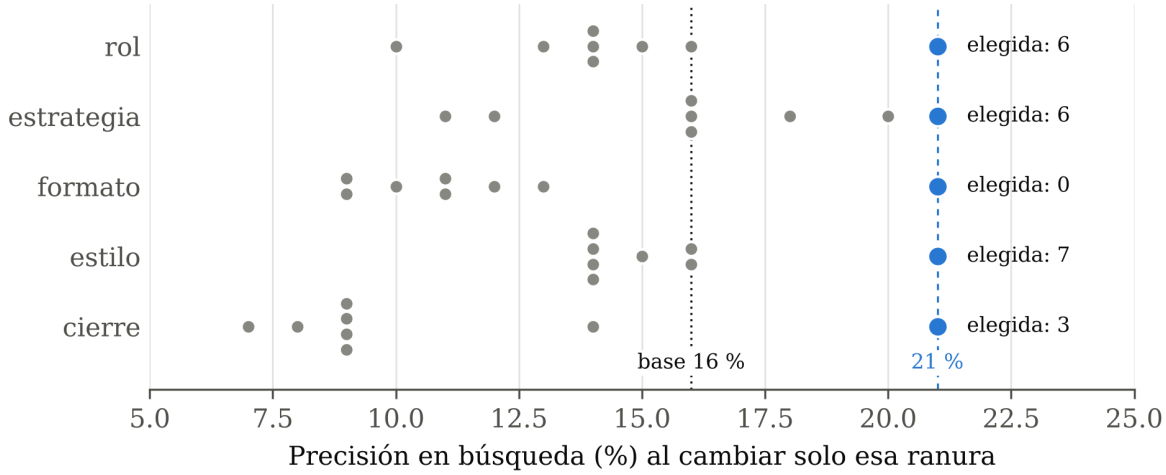

*Nota.* Cada punto es una evaluación sobre 100 instancias; la línea escalonada es la mejor precisión observada hasta ese momento. Las bandas sombreadas marcan las fases 1 y 3.

## Óptimo local verificado

Las 35 configuraciones que difieren de la elegida en una sola ranura fueron evaluadas y ninguna la supera (Figura 4). La más cercana es la estrategia 2, con 20 %; en formato y en cierre todas las alternativas quedan por debajo de la base.

**Figura 4**

*Vecindario de la configuración elegida*




*Nota.* Cada punto gris es una configuración que cambia solo esa ranura respecto a la elegida (en azul, 21 %). Los puntos con la misma precisión se apilan verticalmente.

**Temperatura**

Con $\tau > 0$ la misma configuración da resultados distintos según la semilla (Tabla 8): el azar mueve la precisión entre dos y cuatro puntos. Ningún promedio supera el 21 % de $\tau = 0$, así que se mantiene $\tau = 0.0$, que además es determinista y reproducible.

**Tabla 8**

*Precisión de la configuración elegida según la temperatura*

| Temperatura | Semilla 1 | Semilla 2 | Semilla 3 | Media | Desviación |
|---|---|---|---|---|---|
| 0.0 | 21 % | — | — | 21,0 % | 0,0 |
| 0.3 | 20 % | 17 % | 16 % | 17,7 % | 1,7 |
| 0.7 | 19 % | 18 % | 20 % | 19,0 % | 0,8 |

*Nota.* Con $\tau = 0.0$ la decodificación es voraz y la semilla no influye.

### Configuración final y costo

La Tabla 9 reúne la configuración elegida, sus precisiones, la garantía que ofrece el método y el costo de la búsqueda.

**Tabla 9**

*Resumen de resultados*

| Indicador | Valor |
|---|---|
| Configuración (entrega.json) | rol 6 · estrategia 6 · formato 0 · estilo 7 · cierre 3 · temperatura 0.0 |
| Precisión en búsqueda (100 instancias) | 21 % (base sin instrucciones: 16 %) |
| Precisión en validación (87 instancias) | 18,4 % (16 aciertos) |
| Garantía | Óptimo local 1-opt verificado (35 vecinos); no es un óptimo global demostrado |
| Evaluaciones por fase | Base 1 · cribado 40 · ascenso 17 · verificación 39 · temperatura 6 = 103 |
| Consultas | 10 300, frente a 3 276 800 de la fuerza bruta (0,31 %) |
| Configuraciones completas distintas evaluadas | 56 de 32 768 |
| Tiempo medio por evaluación (T4) | ≈ 181 s (cribado), 197 s (ascenso), 211 s (verificación), 221 s (temperatura) |
| Tiempo total de GPU estimado | ≈ 5,7 h, repartidas en varias sesiones gracias al caché |

### Validación

La configuración elegida se validó con `oraculo.validar` sobre las 87 instancias de validación, que no se usaron durante la búsqueda ni para elegir. Obtuvo 18,4 % (16 aciertos), 2,6 puntos menos que en búsqueda. Con 87 instancias el error estándar es de unos ±4,2 puntos, así que la diferencia está dentro del ruido: la configuración se sostiene con instancias nuevas de las mismas familias. Las familias que más fallan en validación (`last_word_sent`, `repeat_phrase`, `punctuation_dot`) son las mismas que dominaban en búsqueda.

Además, el cuaderno de evaluación que usará la profesora (`evaluacion_G01.ipynb`) se ejecutó en modo de demostración sobre la misma partición: cargó la configuración desde `entrega.json`, calificó las 87 instancias y obtuvo exactamente el mismo 18,4 %, con las mismas familias de fallo. Esto confirma que `entrega.json`, el cuaderno de búsqueda y el de evaluación son coherentes entre sí.

La lectura de los fallos en validación añade tres observaciones. Primero, el modelo a veces enumera o copia las instrucciones del catálogo dentro de la respuesta. Segundo, el rol 6 deja una huella visible: muchas respuestas empiezan con «*I will cover… I will not cover…*», lo que tumba restricciones sobre la primera palabra o el inicio exacto; la mejor configuración no es «limpia», sino la que mejor equilibra daños y ayudas. Tercero, en algunas instancias el modelo repite una frase en bucle, una degeneración conocida de la decodificación voraz en modelos pequeños (Holtzman et al., 2020).

## Discusión y limitaciones

**Óptimo local, no global.** El método garantiza que ningún cambio de una sola ranura mejora la configuración, pero no que no exista una combinación mejor que cambie dos o más ranuras a la vez. La interacción observada entre el rol 6 y la estrategia 6 muestra que esos casos existen. Un óptimo global exigiría el algoritmo exacto, previsto para la siguiente entrega, y el resultado de NP-dureza indica que su costo en el peor caso es exponencial en $m$.

**Ruido de muestreo.** Con 100 instancias, diferencias de uno o dos puntos no distinguen configuraciones (sección Parte 2: costo de evaluación y complejidad de muestreo). La elección del 21 % frente al 20 % de su mejor vecino es la mejor decisión con la evidencia disponible, no una diferencia demostrada.

**Generalización al conjunto privado.** La validación mide la generalización a instancias nuevas de las mismas familias, no a familias nuevas. Además, si el conjunto privado no trae el texto de las restricciones (como ocurre con el formato de IFBench), el cierre 3 no agrega el recordatorio y su ventaja podría desaparecer. El cuaderno de evaluación incluye los verificadores de IFBench para que esas familias se puedan calificar.

**Costo operativo.** La búsqueda tomó cerca de 5,7 horas de GPU y la cuota gratuita de Colab se agotó a mitad de la ejecución. El caché en Drive permitió retomarla sin perder lo generado, pero el tiempo de pared fue mayor que el de cómputo.

**Alcance de esta entrega.** La comparación con búsqueda aleatoria del mismo presupuesto está implementada pero no se ejecutó por costo de GPU. El algoritmo exacto con poda por cota superior, Successive Halving, la comparación de repartos de presupuesto y la validación experimental con instancias sintéticas de tamaño creciente quedan para las entregas siguientes.

## Reproducibilidad

Todos los artefactos están en la rama `main` del repositorio https://github.com/J2517/ADA-2026-2-GP01 (Tabla 10). Los resultados se reproducen sin editar la lógica: el cuaderno de búsqueda reconstruye las 103 evaluaciones desde el caché o las regenera, y el de evaluación carga `entrega.json` y califica un archivo de prueba subido por quien lo ejecuta. Quedan fuera del alcance del equipo los datos privados de la profesora y la calificación final.

**Tabla 10**

*Artefactos del repositorio*

| Archivo | Contenido |
|---|---|
| entrega.json | Grupo, semana y configuración elegida |

| Archivo | Contenido |
|---|---|
| busqueda_G01_resultado.ipynb | Cuaderno de búsqueda ejecutado: fases, trazas, interpretación, curvas, validación |
| busqueda_G01.ipynb | El mismo cuaderno sin salidas, para volver a ejecutarlo |
| evaluacion_G01.ipynb | Evaluación de entrega.json sobre datos privados, sin editar nada |
| registro_busqueda.csv, cribado.csv | Las 103 evaluaciones y el resultado del cribado |
| busqueda.py, registro.py, evaluacion.py | Algoritmo, bitácora y utilidades de evaluación |
| tests/ | 23 pruebas con un oráculo simulado, sin GPU |
| docs/MANUAL_TECNICO.md | Manual técnico: entorno, arquitectura, uso, pruebas y limitaciones |

## Conclusiones

Elegir la configuración de prompt óptima es NP-completo en su versión de decisión, aun con un oráculo exacto: la reducción desde 3-SAT traduce cada cláusula en siete instancias y conserva el número de aciertos, lo que además transfiere el límite de aproximación 7/8 de MAX-3SAT. Con un oráculo costoso y ruidoso, el problema añade una dificultad de muestreo: con 100 instancias no se pueden certificar diferencias de pocos puntos.

La solución base aprovecha la estructura del catálogo. El cribado y las trazas de fallos explican qué daña cada opción y permiten podar el espacio de 32 768 a 243 configuraciones; el ascenso por coordenadas y la verificación del vecindario completo encuentran una configuración con 21 % de precisión en búsqueda y 18,4 % en validación usando el 0,31 % de las consultas de la fuerza bruta. La verificación recuperó una opción que la poda había descartado, una muestra concreta de la interacción entre ranuras que hace difícil el problema y del límite de las decisiones tomadas con opciones aisladas.

El resultado es la mejor configuración observada y un óptimo local verificado. Las próximas entregas compararán este método con el algoritmo exacto y con Successive Halving, y medirán su crecimiento con instancias sintéticas de tamaño creciente.

## Referencias

Agrawal, L. A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., & Khattab, O. (2025). *GEPA: Reflective prompt evolution can outperform reinforcement learning* (arXiv:2507.19457). arXiv. https://arxiv.org/abs/2507.19457

Chang, Y., & Chen, X. (2026). *Naive prompt optimization: Rethinking the need for complex prompt search* (arXiv:2608.27266). arXiv. https://arxiv.org/abs/2608.27266

Cook, S. A. (1971). The complexity of theorem-proving procedures. En *Proceedings of the Third Annual ACM Symposium on Theory of Computing* (pp. 151–158). ACM. https://doi.org/10.1145/800157.805047

DanielMelo404. (2026). *Proyecto-Oraculo-AyD* [Repositorio de software]. GitHub. https://github.com/DanielMelo404/Proyecto-Oraculo-AyD

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.

Håstad, J. (2001). Some optimal inapproximability results. *Journal of the ACM, 48*(4), 798–859. https://doi.org/10.1145/502090.502098

Hoeffding, W. (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association, 58*(301), 13–30. https://doi.org/10.1080/01621459.1963.10500830

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). *The curious case of neural text degeneration* (arXiv:1904.09751). arXiv. https://arxiv.org/abs/1904.09751

Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). Plenum Press.

Pyatkin, V., Malik, S., Graf, V., Ivison, H., Huang, S., Dasigi, P., Lambert, N., & Hajishirzi, H. (2025). *Generalizing verifiable instruction following* (arXiv:2507.02833). arXiv. https://arxiv.org/abs/2507.02833

Qwen Team. (2025). *Qwen3-1.7B* [Modelo de lenguaje]. Hugging Face. https://huggingface.co/Qwen/Qwen3-1.7B

Universidad de Caldas. (2026a). *Proyecto semestral 2026-2: Análisis y diseño de algoritmos* [Documento del curso]. Programa de Ingeniería de Sistemas.

Universidad de Caldas. (2026b). *Primera entrega: qué debe presentar cada grupo. Guía de productos y criterios mínimos de entrega, ADAG1* [Documento del curso]. Programa de Ingeniería de Sistemas.

Wright, S. J. (2015). Coordinate descent algorithms. *Mathematical Programming, 151*(1), 3–34. https://doi.org/10.1007/s10107-015-0892-3

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., & Hou, L. (2023). *Instruction-following evaluation for large language models* (arXiv:2311.07911). arXiv. https://arxiv.org/abs/2311.07911